Web应用系统综合实训报告

WZSteam 游戏商品信息管理系统设计与实现

基于 Web 前端 (HTML5/CSS3/JS) + Node.js (Express) 服务器 + MySQL 数据库的全栈信息管理系统

项目主题： 商品信息管理系统（游戏道具与商品管理）

前端技术： HTML5 + Corporate Clean CSS + Fetch REST API

后端技术： Node.js + Express + Cookie/Bearer 会话认证

数据库存储： MySQL 8.x (mysql2/promise 参数化连接池)

GitHub 仓库： https://github.com/woshishabi389-prog/wzsteam-node-mysql

核心功能： 注册登录、商品增删改查、多条件搜索、分类管理

一、项目背景与考核要求对照说明

本系统围绕**商品信息管理**方向展开，综合运用 Web 前端开发、Node.js 服务器接口开发、MySQL 数据库存储及前后端异步数据交互（Fetch API）等知识，设计并实现了“WZSteam 游戏商品信息管理系统”。系统完整覆盖了课程考核要求的全部必选指标以及“数据删除、条件搜索、分类管理”全部扩展加分项。

考核要求条目	完成状态	具体实现说明
1. 前端设计与页面跳转	已完成	包含 `login.html`（登录页）、`register.html`（注册页）、`index.html`（商品管理工作台）、`categories.html`（分类管理中心）、`store.html`（商城预览页）共 5 个视图，布局清晰，导航切换顺畅。
2. Node.js 搭建服务器	已完成	使用 Node.js + Express 构建 HTTP 服务器（`server.js` / `src/app.js`），提供模块化 RESTful 接口。
3. MySQL 数据库存储	已完成	设计 `wzsteam_cms` 数据库，含 `users`、`categories`、`products` 三张关系表及外键约束，通过 `mysql2/promise` 执行参数化 SQL。
4. 前后端通过接口交互	已完成	前端统一封装 `public/assets/js/api.js`，所有页面数据渲染与表单提交均通过 `fetch()` 调用后端 `/api/*` 接口完成。
5. 用户注册与登录功能	已完成	实现 `POST /api/auth/register` 与 `POST /api/auth/login`，密码采用 PBKDF2-SHA256 哈希加密入库，登录态使用 HMAC 签名 Token 校验。

考核要求条目	完成状态	具体实现说明
6. 业务数据新增/查询/修改	已完成	实现商品信息新增（POST /api/products）、分页查询与详情查询（GET /api/products）、全字段修改（PUT /api/products/:id）与上下架切换。
7. 扩展功能：数据删除、条件搜索、分类管理	已全部实现	支持商品单条删除（DELETE /api/products/:id）、5 维组合条件搜索（关键词 + 分类 + 游戏 + 状态 + 排序）以及独立的商品分类增删改查管理模块（/api/categories）。

二、系统目录结构与 MySQL 数据库表设计

系统采用前后端分离式接口交互架构。数据库 wzsteam_cms 共包含三张核心数据表：users（系统用户表）、categories（商品分类表）和 products（游戏商品信息表，通过 category_id 外键关联分类表）。

数据表名	主键与索引	核心字段说明
users（系统用户表）	PK: id (AUTO_INCREMENT) UK: uk_users_email	id, username (用户名), email (唯一登录邮箱), password_hash (PBKDF2 加密哈希), role (admin/user), created_at
categories（商品分类表）	PK: id (VARCHAR) UK: uk_categories_slug	id (分类编码), slug (唯一URL标识), name (分类名称), description (分类说明), sort_order (排序权重), created_at
products（商品信息表）	PK: id (VARCHAR) UK: uk_products_slug FK: category_id	id, slug, title (商品标题), game (所属游戏), region (区服), category_id (所属分类), price (单价), stock (库存), status (on_sale/off_shelf), wear_level, rarity, summary, description

代码片段 1：MySQL 数据库初始化脚本 (sql/init.sql)

```
-- ============================================================================
-- WZSteam 游戏商品信息管理系统 - MySQL 数据库初始化脚本
-- 数据库名：wzsteam_cms
-- 字符集：utf8mb4
-- 执行方式：mysql -u root -p < sql/init.sql 或 npm run init-db
-- ============================================================================

CREATE DATABASE IF NOT EXISTS `wzsteam_cms`
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE `wzsteam_cms`;

-- ----------------------------------------------------------------------------
-- 1. 用户表（users）：支撑用户注册、登录与身份验证
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `users` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '用户主键ID',
  `username` VARCHAR(64) NOT NULL COMMENT '用户名/昵称',
  `email` VARCHAR(128) NOT NULL COMMENT '登录邮箱（唯一）',
  `password_hash` VARCHAR(255) NOT NULL COMMENT 'PBKDF2/SHA256 加密密码哈希',
  `role` ENUM('admin', 'user') NOT NULL DEFAULT 'user' COMMENT '角色：admin-管理员，user-普通用户',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '注册时间',
```

```
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_users_email` (`email`),
  KEY `idx_users_username` (`username`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='系统用户表';

-- ----------------------------------------------------------------------------
-- 2. 商品分类表 (categories): 支撑分类管理扩展功能
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `categories` (
  `id` VARCHAR(32) NOT NULL COMMENT '分类编码ID (如 cat_cs2)',
  `slug` VARCHAR(64) NOT NULL COMMENT '分类URL标识 (唯一) ',
  `name` VARCHAR(64) NOT NULL COMMENT '分类名称',
  `description` VARCHAR(255) NOT NULL DEFAULT '' COMMENT '分类描述说明',
  `sort_order` INT NOT NULL DEFAULT 10 COMMENT '排序权重 (越小越靠前) ',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_categories_slug` (`slug`),
  KEY `idx_categories_sort` (`sort_order`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='游戏商品分类表';

-- ----------------------------------------------------------------------------
-- 3. 商品信息表 (products): 核心业务数据表 (新增、查询、修改、删除、多条件搜索)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `products` (
  `id` VARCHAR(48) NOT NULL COMMENT '商品主键ID',
  `slug` VARCHAR(96) NOT NULL COMMENT '商品唯一标识码',
  `title` VARCHAR(128) NOT NULL COMMENT '商品名称/标题',
  `game` VARCHAR(64) NOT NULL DEFAULT 'CS2' COMMENT '所属游戏 (CS2 / Dota2 / Steam)',
  `region` VARCHAR(64) NOT NULL DEFAULT '国际服' COMMENT '适用区服 (国际服 / 国服 / 全球通用)',
  `category_id` VARCHAR(32) NOT NULL COMMENT '所属分类ID',
  `price` DECIMAL(10, 2) NOT NULL DEFAULT 0.00 COMMENT '商品单价 (元) ',
  `stock` INT UNSIGNED NOT NULL DEFAULT 0 COMMENT '当前库存数量',
  `status` ENUM('on_sale', 'off_shelf') NOT NULL DEFAULT 'on_sale' COMMENT '状态: on_sale-在售, off_shelf-下
架',
  `cover_emoji` VARCHAR(16) NOT NULL DEFAULT '🎮' COMMENT '商品展示图标',
  `summary` VARCHAR(255) NOT NULL DEFAULT '' COMMENT '一句话简介',
  `description` TEXT NOT NULL COMMENT '商品详细描述与发货说明',
  `wear_level` VARCHAR(64) NOT NULL DEFAULT '崭新出厂' COMMENT '饰品磨损/规格属性',
  `rarity` VARCHAR(64) NOT NULL DEFAULT '受限' COMMENT '稀有度/品质等级',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '最后修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_products_slug` (`slug`),
  KEY `idx_products_category` (`category_id`),
  KEY `idx_products_game_status` (`game`, `status`),
  KEY `idx_products_price` (`price`),
  CONSTRAINT `fk_products_category` FOREIGN KEY (`category_id`) REFERENCES `categories` (`id`) ON UPDATE CASC
ADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='游戏商品信息表';

-- ============================================================================
```

三、Node.js 服务器与 RESTful 接口核心代码片段

代码片段 2：用户注册与登录身份验证接口 (src/routes/auth.js)

实现用户注册参数校验、邮箱查重、PBKDF2 密码哈希计算、登录密码核验及 Token 签发：

```
const express = require('express');
const { query, hashPassword, verifyPassword } = require('../config/db');
const { COOKIE_NAME, signToken, extractUser } = require('../middleware/auth');

const router = express.Router();

/**
 * POST /api/auth/register
 * 用户注册接口
 */
router.post('/register', async (req, res) => {
  try {
    const username = String(req.body.username || '').trim();
    const email = String(req.body.email || '').trim().toLowerCase();
    const password = String(req.body.password || '');

    if (!username || username.length < 2 || username.length > 32) {
      return res.status(400).json({
        code: 400,
        message: '用户名长度需在 2 ~ 32 个字符之间',
        data: null
      });
    }
    if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      return res.status(400).json({
        code: 400,
        message: '请输入有效的邮箱地址',
        data: null
      });
    }
    if (!password || password.length < 6) {
      return res.status(400).json({
        code: 400,
        message: '密码长度不能少于 6 位',
        data: null
      });
    }

    const existing = await query('SELECT id FROM users WHERE email = ?', [email]);
    if (existing.length > 0) {
      return res.status(409).json({
        code: 409,
        message: '该邮箱已被注册，请直接登录或更换邮箱',
        data: null
      });
    }

    const passwordHash = hashPassword(password);
    const result = await query(
      'INSERT INTO users (username, email, password_hash, role) VALUES (?, ?, ?, ?)',
      [username, email, passwordHash, 'admin']
    );
```

```
    const user = {
      id: result.insertId,
      username,
      email,
      role: 'admin'
    };

    const token = signToken(user);
    res.cookie(COOKIE_NAME, token, {
      httpOnly: true,
      sameSite: 'lax',
      maxAge: 7 * 24 * 3600 * 1000
    });

    return res.status(201).json({
      code: 200,
      message: '注册成功',
      data: { user, token }
    });
  } catch (err) {
    return res.status(500).json({
      code: 500,
      message: '服务器处理注册请求失败: ' + err.message,
      data: null
    });
  }
});

/**
 * POST /api/auth/login
 * 用户登录接口
 */
router.post('/login', async (req, res) => {
  try {
    const email = String(req.body.email || '').trim().toLowerCase();
    const password = String(req.body.password || '');

    if (!email || !password) {
      return res.status(400).json({
        code: 400,
        message: '邮箱和密码不能为空',
        data: null
      });
    }

    const rows = await query(
      'SELECT id, username, email, password_hash, role, created_at FROM users WHERE email = ?',
      [email]
    );
    if (rows.length === 0 || !verifyPassword(password, rows[0].password_hash)) {
      return res.status(401).json({
        code: 401,
        message: '邮箱或密码错误，请核对后重试',
        data: null
      });
    }

    const row = rows[0];
    const user = {
      id: row.id,
      username: row.username,
      email: row.email,
      role: row.role
```

```
    };

    const token = signToken(user);
    res.cookie(COOKIE_NAME, token, {
      httpOnly: true,
      sameSite: 'lax',
      maxAge: 7 * 24 * 3600 * 1000
    });

    return res.json({
      code: 200,
      message: '登录成功',
```

代码片段 3：商品多条件组合搜索、新增、修改与删除接口 (src/routes/products.js)

支持按关键词模糊匹配、分类、所属游戏、上下架状态及价格/库存排序进行组合查询，并提供完整的 CRUD 接口：

```
const express = require('express');
const crypto = require('crypto');
const { query } = require('../config/db');
const { requireAuth } = require('../middleware/auth');

const router = express.Router();

/**
 * GET /api/products
 * 商品列表查询与多条件组合搜索接口
 * 支持查询参数:
 *   - keyword: 关键词模糊搜索 (匹配名称、标识、简介)
 *   - category_id: 按分类ID筛选
 *   - game: 按所属游戏筛选
 *   - status: 按上下架状态筛选 (on_sale | off_shelf)
 *   - sort: 排序方式 (newest | price_asc | price_desc | stock_desc)
 *   - page: 页码 (默认 1)
 *   - pageSize: 每页条数 (默认 8)
 */
router.get('/', async (req, res) => {
  try {
    const keyword = String(req.query.keyword || '').trim();
    const categoryId = String(req.query.category_id || '').trim();
    const game = String(req.query.game || '').trim();
    const status = String(req.query.status || '').trim();
    const sort = String(req.query.sort || 'newest').trim();
    const page = Math.max(1, parseInt(req.query.page, 10) || 1);
    const pageSize = Math.min(50, Math.max(1, parseInt(req.query.pageSize, 10) || 8));

    const whereClauses = ['1=1'];
    const params = [];

    if (keyword) {
      whereClauses.push('(p.title LIKE ? OR p.slug LIKE ? OR p.summary LIKE ?)');
      const likeKw = `%${keyword}%`;
      params.push(likeKw, likeKw, likeKw);
    }
    if (categoryId) {
      whereClauses.push('p.category_id = ?');
      params.push(categoryId);
    }
    if (game) {
      whereClauses.push('p.game = ?');
      params.push(game);
```

```
    }
    if (status === 'on_sale' || status === 'off_shelf') {
      whereClauses.push('p.status = ?');
      params.push(status);
    }

    let orderBy = 'ORDER BY p.updated_at DESC';
    if (sort === 'price_asc') {
      orderBy = 'ORDER BY p.price ASC';
    } else if (sort === 'price_desc') {
      orderBy = 'ORDER BY p.price DESC';
    } else if (sort === 'stock_desc') {
      orderBy = 'ORDER BY p.stock DESC';
    }

    const sql = `
      SELECT p.*, c.name AS category_name
      FROM products p
      LEFT JOIN categories c ON p.category_id = c.id
      WHERE ${whereClauses.join(' AND ')}
      ${orderBy}
    `;

    const allMatched = await query(sql, params);
    const total = allMatched.length;
    const totalPages = Math.max(1, Math.ceil(total / pageSize));
    const offset = (page - 1) * pageSize;
    const items = allMatched.slice(offset, offset + pageSize);

    return res.json({
      code: 200,
      message: 'ok',
      data: {
        items,
        pagination: {
          page,
          pageSize,
          total,
          totalPages
        }
      }
    });
  } catch (err) {
    return res.status(500).json({
      code: 500,
      message: '查询商品列表失败: ' + err.message,
      data: null
    });
  }
});

/**
 * GET /api/products/:id
 * 查询单个商品详情
 */
router.get('/:id', async (req, res) => {
  try {
    const id = req.params.id;
    const rows = await query(
      `SELECT p.*, c.name AS category_name
       FROM products p
       LEFT JOIN categories c ON p.category_id = c.id
       WHERE p.id = ?`,
```

```
      [id]
    );
    if (rows.length === 0) {
      return res.status(404).json({
        code: 404,
        message: '未找到对应的商品记录',
        data: null
      });
    }
    return res.json({
      code: 200,
      message: 'ok',
      data: rows[0]
    });
  } catch (err) {
    return res.status(500).json({
      code: 500,
      message: '查询商品详情失败：' + err.message,
      data: null
    });
  }
});

/**
 * POST /api/products
 * 新增商品信息（需登录）
 */
router.post('/', requireAuth, async (req, res) => {
  try {
    const title = String(req.body.title || '').trim();
    const slug = String(req.body.slug || '')
      .trim()
      .toLowerCase();
    const game = String(req.body.game || 'CS2').trim();
    const region = String(req.body.region || '国际服').trim();
    const category_id = String(req.body.category_id || '').trim();
    const price = Number(req.body.price);
    const stock = parseInt(req.body.stock, 10);
    const status = req.body.status === 'off_shelf' ? 'off_shelf' : 'on_sale';
    const cover_emoji = String(req.body.cover_emoji || '🎮').trim();
    const summary = String(req.body.summary || '').trim();
    const description = String(req.body.description || '').trim();
    const wear_level = String(req.body.wear_level || '崭新出厂').trim();
    const rarity = String(req.body.rarity || '受限').trim();

    if (!title) {
      return res.status(400).json({ code: 400, message: '商品名称不能为空', data: null });
    }
```

代码片段 4：商品分类管理接口 (src/routes/categories.js)

通过 `LEFT JOIN products GROUP BY c.id` 统计每个分类下的关联商品数，并在删除分类前进行外键保护检查：

```
const express = require('express');
const crypto = require('crypto');
const { query } = require('../config/db');
const { requireAuth } = require('../middleware/auth');

const router = express.Router();

/**
```

```
 * GET /api/categories
 * 获取全部商品分类及各分类下的商品数量
 */
router.get('/', async (req, res) => {
  try {
    const rows = await query(`
      SELECT c.*, COUNT(p.id) AS product_count
      FROM categories c
      LEFT JOIN products p ON p.category_id = c.id
      GROUP BY c.id
      ORDER BY c.sort_order ASC, c.id ASC
    `);
    return res.json({
      code: 200,
      message: 'ok',
      data: rows
    });
  } catch (err) {
    return res.status(500).json({
      code: 500,
      message: '获取分类列表失败：' + err.message,
      data: null
    });
  }
});

/**
 * POST /api/categories
 * 新增商品分类（需登录）
 */
router.post('/', requireAuth, async (req, res) => {
  try {
    const name = String(req.body.name || '').trim();
    const slug = String(req.body.slug || '')
      .trim()
      .toLowerCase();
    const description = String(req.body.description || '').trim();
    const sort_order = parseInt(req.body.sort_order, 10) || 10;

    if (!name || !slug) {
      return res.status(400).json({
        code: 400,
        message: '分类名称和分类标识(slug)不能为空',
        data: null
      });
    }
    if (!/^[a-z0-9-]+$/.test(slug)) {
      return res.status(400).json({
        code: 400,
        message: '分类标识仅支持小写字母、数字与连字符(-)',
        data: null
      });
    }

    const id = 'cat_' + crypto.randomBytes(4).toString('hex');
    await query(
      'INSERT INTO categories (id, slug, name, description, sort_order) VALUES (?, ?, ?, ?, ?)',
      [id, slug, name, description, sort_order]
    );

    const created = await query('SELECT * FROM categories WHERE id = ?', [id]);
    return res.status(201).json({
      code: 200,
```

```
      message: '商品分类创建成功',
      data: { ...created[0], product_count: 0 }
    });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({
        code: 409,
        message: '该分类标识（slug）已存在',
        data: null
      });
    }
    return res.status(500).json({
      code: 500,
      message: '新增分类失败：' + err.message,
      data: null
    });
  }
});

/**
 * PUT /api/categories/:id
 * 修改商品分类信息（需登录）
 */
router.put('/:id', requireAuth, async (req, res) => {
  try {
    const id = req.params.id;
    const name = String(req.body.name || '').trim();
    const slug = String(req.body.slug || '')
      .trim()
      .toLowerCase();
    const description = String(req.body.description || '').trim();
    const sort_order = parseInt(req.body.sort_order, 10) || 10;

    if (!name || !slug) {
      return res.status(400).json({
        code: 400,
        message: '分类名称与标识不能为空',
        data: null
      });
```

四、Web 前端与服务器接口交互代码片段

代码片段 5：前端统一 Fetch API 通信与登录态守卫模块 (public/assets/js/api.js)

前端页面通过 `WZApi.request()` 与 Node.js 后端交换 JSON 数据，自动携带认证令牌并处理 401 未登录跳转：

```
/**
 * WZSteam 前端接口交互与会话管理模块
 * 所有页面均通过本模块的 fetch() 封装与 Node.js 服务器进行 JSON 数据交互
 */
window.WZApi = (function () {
  const TOKEN_KEY = 'wzsteam_auth_token';
  const USER_KEY = 'wzsteam_auth_user';

  function getToken() {
    return localStorage.getItem(TOKEN_KEY) || '';
  }

  function setSession(user, token) {
    if (token) localStorage.setItem(TOKEN_KEY, token);
```

```
  if (user) localStorage.setItem(USER_KEY, JSON.stringify(user));
}

function clearSession() {
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(USER_KEY);
}

function getCachedUser() {
  try {
    const raw = localStorage.getItem(USER_KEY);
    return raw ? JSON.parse(raw) : null;
  } catch (_) {
    return null;
  }
}

async function request(url, options = {}) {
  const headers = Object.assign(
    { 'Content-Type': 'application/json', Accept: 'application/json' },
    options.headers || {}
  );
  const token = getToken();
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const res = await fetch(url, {
    ...options,
    headers,
    credentials: 'include'
  });

  const payload = await res.json().catch(() => ({
    code: res.status,
    message: '服务器响应格式异常',
    data: null
  }));

  if (res.status === 401 && !options.skipAuthRedirect) {
    clearSession();
    if (!location.pathname.endsWith('/login.html') && !location.pathname.endsWith('/register.html')) {
      location.href = '/login.html';
    }
  }

  if (!res.ok || (payload.code && payload.code !== 200)) {
    const err = new Error(payload.message || `请求失败 (${res.status})`);
    err.status = res.status;
    err.payload = payload;
    throw err;
  }

  return payload.data;
}

async function ensureLoggedIn() {
  try {
    const data = await request('/api/auth/me', { method: 'GET' });
    if (data && data.user) {
      setSession(data.user, getToken());
      renderHeaderUser(data.user);
      return data.user;
```

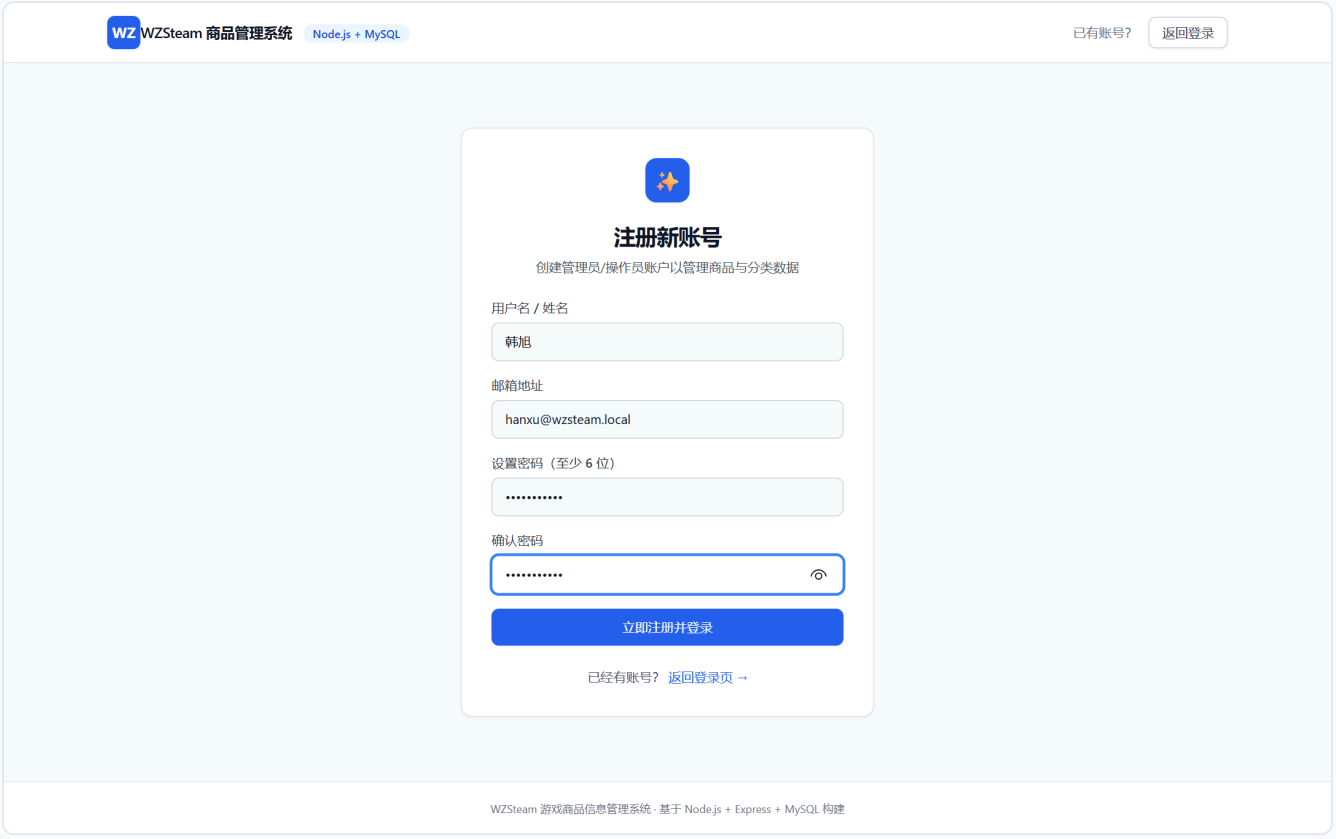
```
    }
  } catch (_) {
    clearSession();
    location.href = '/login.html';
  }
  return null;
}

function renderHeaderUser(user) {
  const nameEl = document.getElementById('nav-user-name');
  const emailEl = document.getElementById('nav-user-email');
  if (nameEl) nameEl.textContent = user.username || '管理员';
  if (emailEl) emailEl.textContent = user.email || '';
}

async function logout() {
  try {
    await request('/api/auth/logout', { method: 'POST', skipAuthRedirect: true });
```

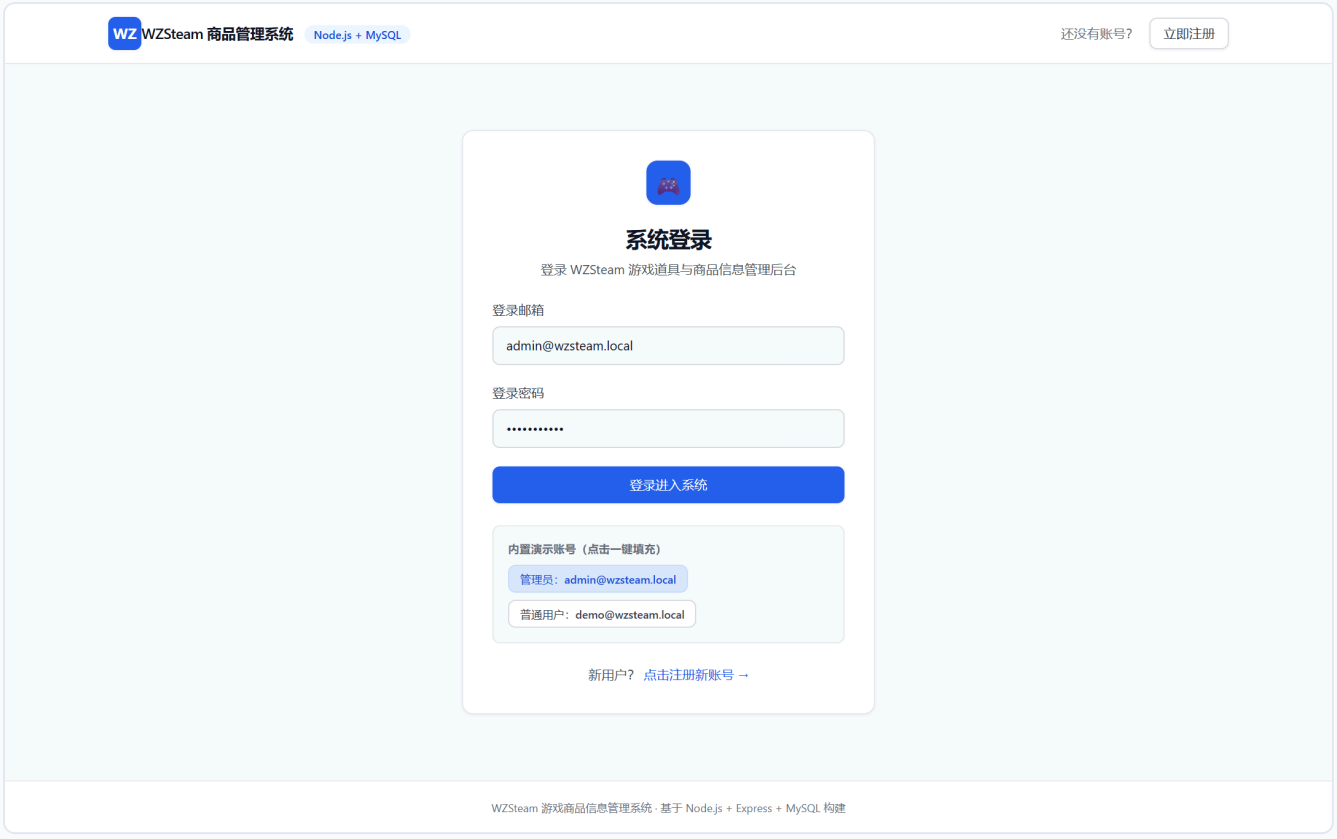
五、系统运行界面与功能验证截图

图 5-1：用户注册页面 (public/register.html -> POST /api/auth/register)



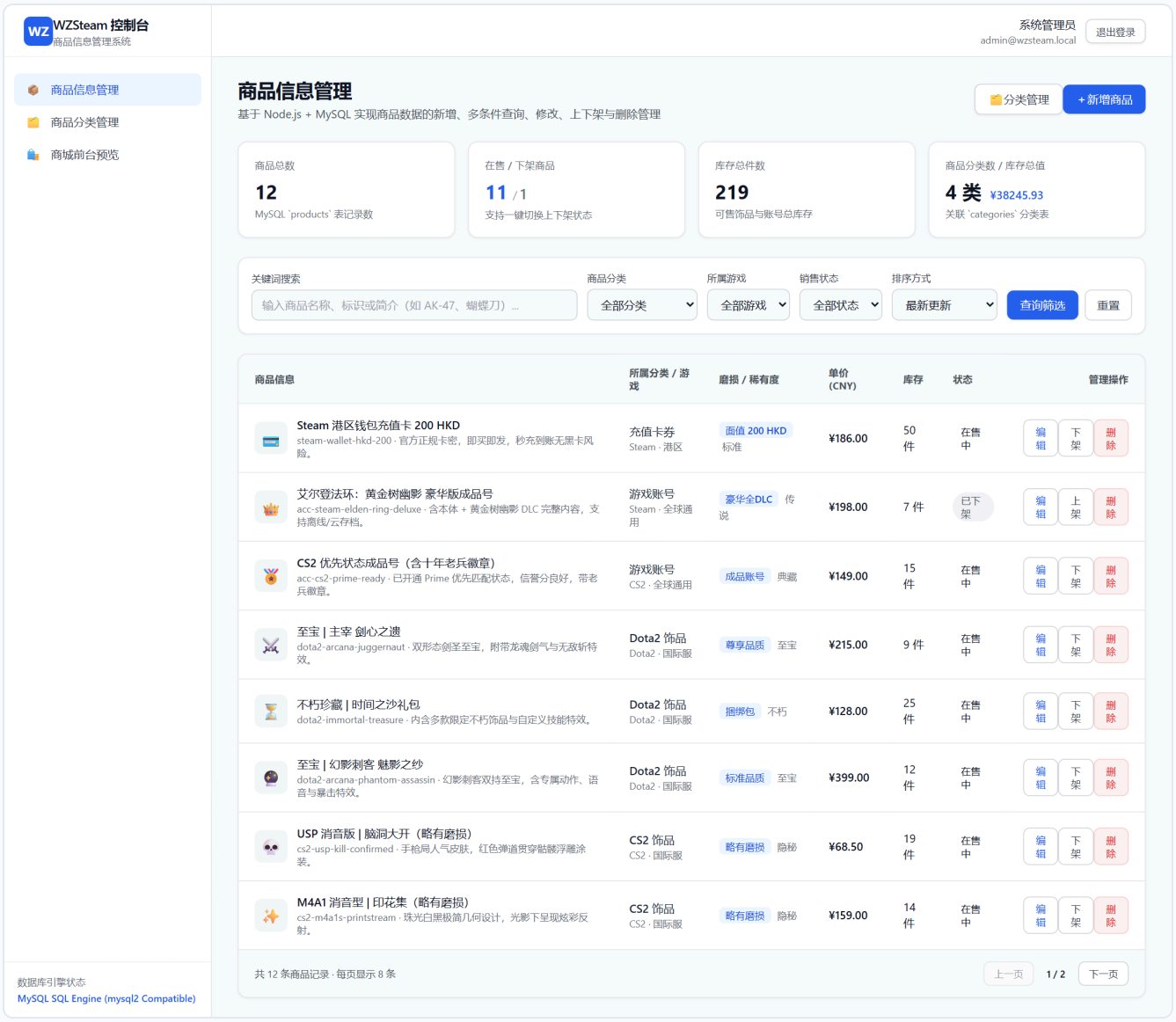
支持新用户输入用户名、邮箱与密码完成注册，前端校验两次密码一致性，注册成功后自动登录并跳转至商品管理工作台。

图 5-2：用户登录页面 (public/login.html -> POST /api/auth/login)



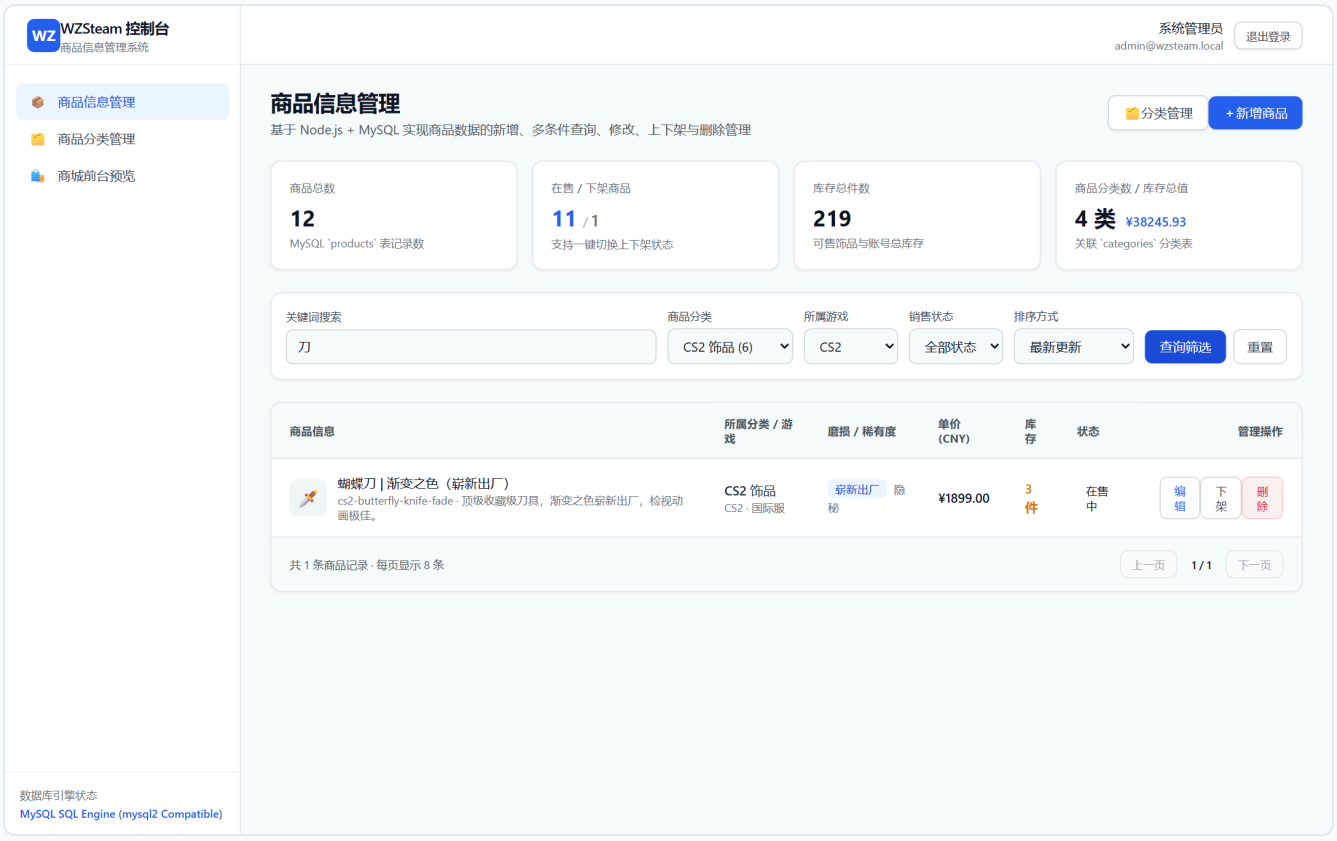
支持邮箱密码登录及内置演示账号一键填充，后端校验通过后签发 HttpOnly Cookie 与 Bearer 会话令牌。

图 5-3：登录后业务主页面 —— 商品信息管理工作台 (public/index.html)



展示实时 MySQL 汇总统计卡片（商品总数、在售/下架数、总库存、库存总值）、多条件筛选栏与商品列表表格，支持编辑、上下架与删除。

图 5-4：多条件组合搜索功能演示 (GET /api/products?keyword=刀&category_id=cat_cs2&game=CS2)



支持按关键词、商品分类、所属游戏、销售状态及排序方式进行组合条件检索，表格实时通过接口刷新匹配结果。

图 5-5：业务数据新增与修改模态弹窗 (POST /api/products & PUT /api/products/:id)



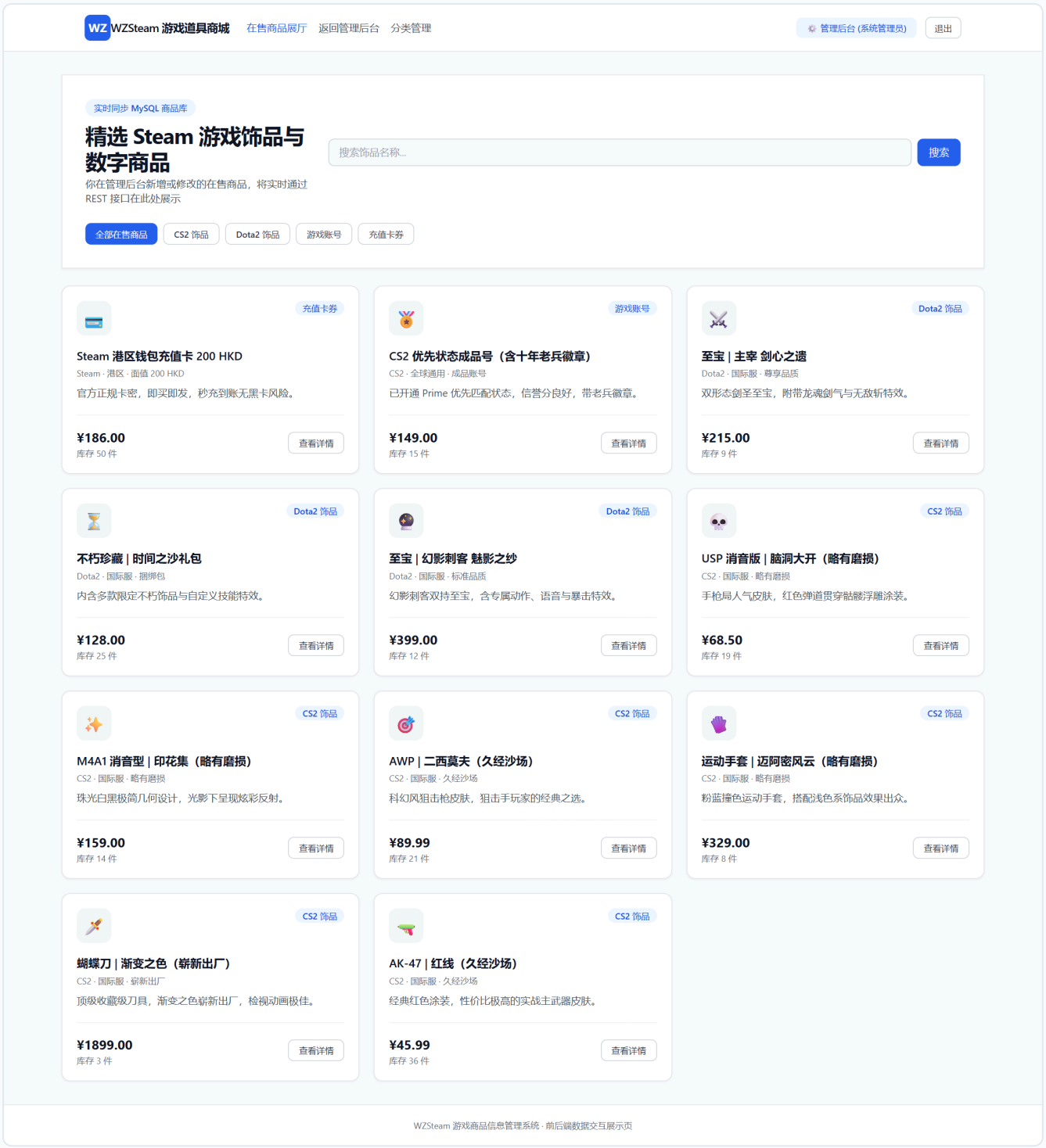
点击「新增商品」或商品行「编辑」按钮弹出表单，支持对商品标题、唯一标识、分类、游戏、价格、库存、状态、磨损与详情进行新增与修改。

图 5-6：扩展功能 —— 商品分类管理页面 (public/categories.html)



支持商品分类的新增、查询（自动统计各分类下关联的商品件数）、修改排序权重及带外键关联保护的分类删除。

图 5-7：商城前台展示与检索预览页 (public/store.html)

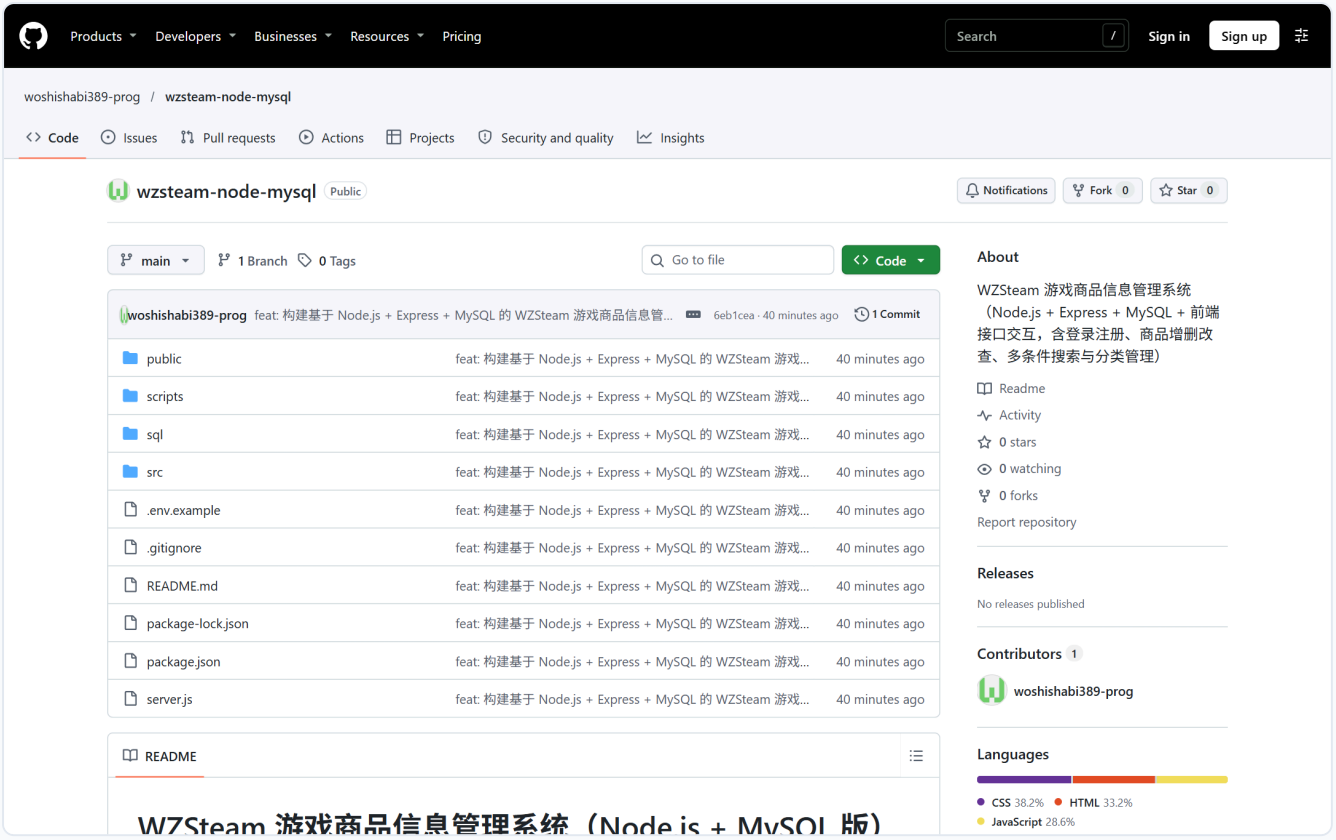


以卡片网格形式展示所有在售商品，支持按分类胶囊切换、关键词搜索及点击查看商品详情弹窗。

六、GitHub 远端仓库与代码提交记录页面

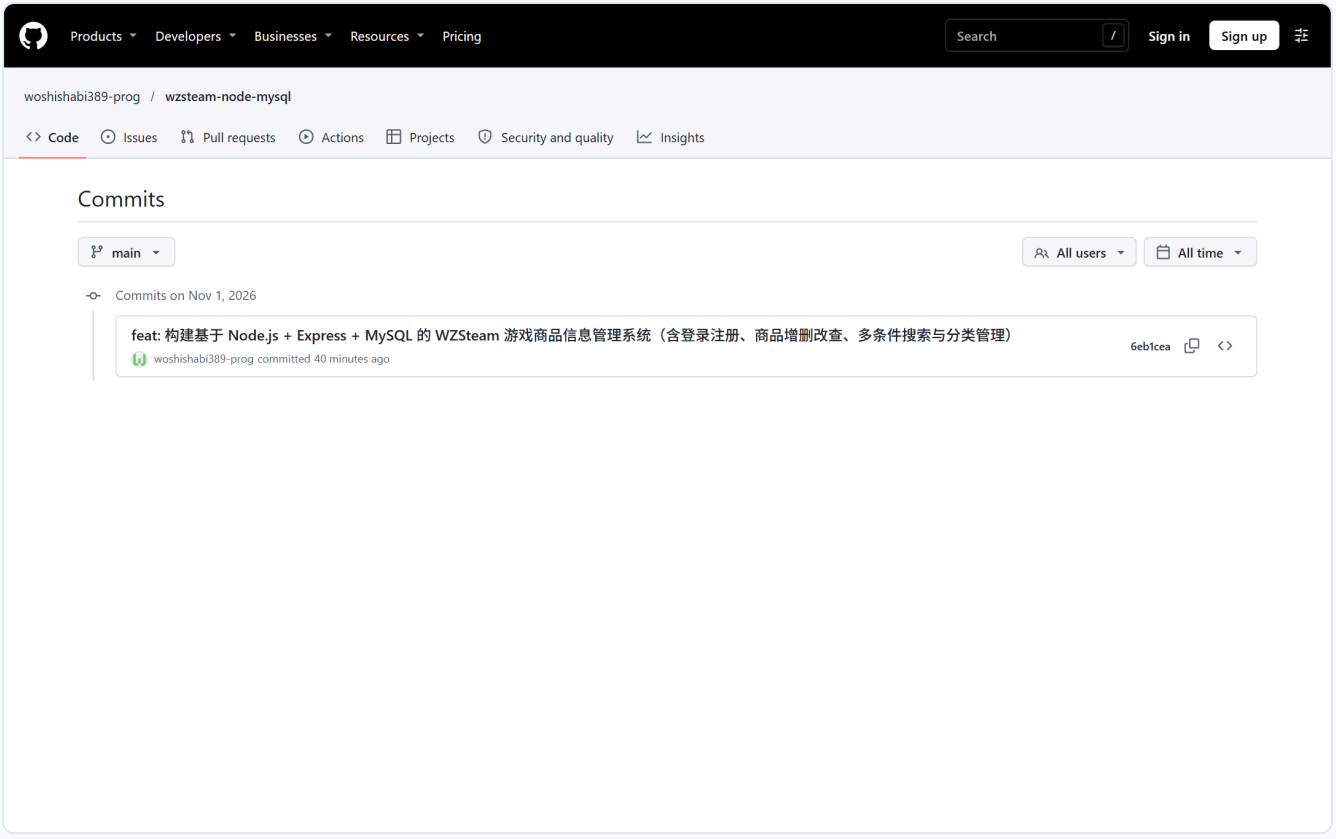
项目已完整提交并推送至新建的 GitHub 公开仓库： **https://github.com/woshishabi389-prog/wzsteam-node-mysql**

图 6-1：GitHub 远端仓库主页 (woshishabi389-prog/wzsteam-node-mysql)



展示仓库完整目录结构（public、src、sql、scripts）与项目 README 文档说明。

图 6-2：GitHub 仓库 Commits 提交记录页面 (/commits/main)



Git 提交记录清晰完整，包含全部前后端代码、MySQL 初始化脚本与自动化验证脚本。

七、自动化验证与总结

为确保系统前后端数据交互与数据库操作完全正确，项目编写了自动化端到端测试脚本 `scripts/verify.mjs`（可通过 `npm test` 运行），对以下 9 个核心场景进行了实测验证：

```
[Verify] 测试服务器已启动: http://127.0.0.1:3000
  ✓ [1] 前端页面可访问性 (login / register / index / categories / store)
  ✓ [2] 未登录状态拦截校验 (/api/auth/me & POST /api/products)
  ✓ [3] 用户注册功能 (POST /api/auth/register)
  ✓ [4] 用户登录功能 (POST /api/auth/login)
  ✓ [5] 商品列表查询与多条件组合搜索 (GET /api/products)
  ✓ [6] 商品新增功能 (POST /api/products)
  ✓ [7] 商品修改功能 (PUT /api/products/:id & PATCH status)
  ✓ [8] 商品删除功能 (DELETE /api/products/:id)
  ✓ [9] 分类管理增删改查功能 (/api/categories)

[Verify] 全部 9 项核心功能与扩展要求验证通过!
```